KANT AX/AI 엔지니어 트랙 · 33

Docker 3

MySQL 컨테이너로 데이터베이스 쓰기

주문을 표에 담고, 지워도 남게 해요



오늘의 순서

1	**MySQL 컨테이너 켜기** 공식 이미지, -e 환경 변수, logs, exec로 접속	5–9쪽
2	**데이터베이스와 표 만들기** CREATE DATABASE, USE, CREATE TABLE, DESCRIBE, INSERT, SELECT, DROP	10–16쪽
3	**계정과 접속 방법** root와 앱 계정, bash로 들어가기, -e로 한 문장만	17–19쪽
4	**볼륨과 compose** 지우면 날아가는 표, /var/lib/mysql에 볼륨, compose.yaml	20–25쪽
5	**자주 나는 오류** 비밀번호, 준비 중, 포트 겹침, SQL 실수, 볼륨 함정	26–31쪽
6	**실제 상황 예제** 다음 날에도 남는 주문, SQL 파일로 메뉴 채우기	32–33쪽

3–4쪽은 들어가기, 34–35쪽은 막힐 때와 한 장 요약이에요.

들어가기

MySQL을 직접 설치하면 켜고 지우고 버전 맞추기가 번거로워요

컴퓨터에 직접 설치

1 설치 파일을 받아 설치 마법사를 넘겨요

2 컴퓨터를 켤 때마다 뒤에서 늘 돌아요

3 지우려면 프로그램과 데이터 폴더를 따로 찾아요

4 프로젝트마다 다른 버전을 쓰기 어려워요

Docker 컨테이너

1 `docker run` 한 줄로 켜요

2 쓸 때만 켜고 `docker stop`으로 꺼요

3 `docker rm`으로 깨끗이 지워요

4 `mysql:8.4`처럼 태그로 버전을 골라요

32에서 읽어 본 표와 SQL을 오늘은 컨테이너로 켠 MySQL에서 직접 쳐 봐요.

실제 회사에서도 개발할 때는 데이터베이스를 컨테이너로 띄워 쓰는 경우가 많아요.

들어가기

MySQL 서버는 컨테이너 안에서 돌고, 데이터는 볼륨에 남아요



컨테이너를 지워도 표는 볼륨에 남아서, 다시 켜면 그대로예요.

1. MySQL 컨테이너 켜기

Docker Hub의 공식 이미지 mysql:8.4를 받아요

```
~ $ docker pull mysql:8.4
8.4: Pulling from library/mysql
Digest: sha256:a81d043930e5…
Status: Image is up to date for mysql:8.4
docker.io/library/mysql:8.4

~ $ docker images mysql
IMAGE        ID             DISK USAGE   CONTENT SIZE   EXTRA
mysql:8.4    a81d043930e5       1.12GB          254MB
```

29에서 배운 pull

이미지가 1GB가 넘어서 처음 받을 때는 몇 분 걸릴 수 있어요.

Docker Official Image

29에서 본 공식 마크가 붙은 이미지예요. Docker Hub에서 mysql을 검색하면 맨 위에 나와요.

태그 8.4

오래 고쳐 주는 LTS 버전이에요. latest 대신 버전을 적어 두면 다음 달에 받아도 같은 MySQL이 와요.

1. MySQL 컨테이너 켜기

docker run 한 줄로 MySQL 서버를 켜요

```
~ $ docker run -d --name db -e MYSQL_ROOT_PASSWORD=cafe1234 -e MYSQL_DATABASE=cafe -p 3306:3306 mysql:8.4
f761cbba159d…
```

`docker run -d --name db` — 29에서 배운 대로 뒤에서 켜고 이름은 db

`-e MYSQL_ROOT_PASSWORD=cafe1234` — 관리자(root) 비밀번호 · 꼭 있어야 해요

`-e MYSQL_DATABASE=cafe` — 처음 켤 때 cafe 데이터베이스를 만들어요

`-p 3306:3306` — MySQL이 기다리는 3306번 포트를 열어요

`mysql:8.4` — 받아 둔 이미지

-e 두 개만 새로 나왔어요. 다음 장에서 봐요.

1. MySQL 컨테이너 켜기

-e로 컨테이너에 설정값(환경 변수)을 건네요

```
~ $ docker exec db env
PATH=…
HOSTNAME=f761cbba159d
MYSQL_DATABASE=cafe
MYSQL_ROOT_PASSWORD=cafe1234   -e로 건넨 값
GOSU_VERSION=1.19
MYSQL_MAJOR=8.4
MYSQL_VERSION=8.4.13-1.el9
MYSQL_SHELL_VERSION=8.4.10-1.el9
HOME=/root
```

환경 변수는 프로그램이 켜질 때 읽는 이름=값 쪽지예요.

MySQL 이미지는 처음 켜질 때 이 쪽지를 읽고 비밀번호와 데이터베이스를 만들어요. 쓸 수 있는 이름은 Docker Hub의 mysql 설명 페이지 Environment Variables에 적혀 있어요.

> **cafe1234는 연습용이에요**
>
> 실제 서비스는 긴 비밀번호를 쓰고, 비밀번호를 적은 파일은 GitHub에 올리지 않아요.

1. MySQL 컨테이너 켜기

logs에 port: 3306이 찍히면 다 켜진 거예요

```
~ $ docker logs db
…
[Note] [Entrypoint]: Initializing database files
[Note] [Entrypoint]: Creating database cafe
[Note] [Entrypoint]: MySQL init process done. Ready for start up.
…
… ready for connections. Version: '8.4.13'  socket: '/var/run/mysqld/mysqld.sock'  port: 0  MySQL Community Server - GPL.
… ready for connections. Version: '8.4.13'  socket: '/var/run/mysqld/mysqld.sock'  port: 3306  MySQL Community Server - GPL.
```

1 데이터 파일을 만들어요

2 잠깐 켠 임시 서버(port: 0)로 cafe를 만들어요

3 진짜 서버를 3306번 포트로 켜요

처음 켤 때는 10초쯤 걸려요. 그 전에 들어가면 오류가 나요. 5파트에서 봐요.

1. MySQL 컨테이너 켜기

docker exec -it로 컨테이너 안의 mysql 프로그램에 들어가요

```
~ $ docker exec -it db mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
…
Server version: 8.4.13 MySQL Community Server - GPL
…
mysql> exit
Bye
~ $
```

cafe1234를 쳐요. 화면에 안 보여요

SQL을 칠 차례예요

내 터미널로 돌아왔어요

30에서는

`exec -it web bash`

오늘은

`exec -it db mysql`

bash 대신 MySQL에 접속하는 프로그램 mysql을 열어요.

`-u root -p`

-u는 사용자 이름, -p는 비밀번호를 물어보라는 뜻이에요.

SHOW DATABASES;로 서버 안의 데이터베이스를 봐요

```
~ $ docker exec -it db mysql -u root -p
Enter password:
…
mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| cafe               |
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.01 sec)
```

문장 끝에 ;

-e MYSQL_DATABASE로 만든 것

MySQL이 스스로 쓰는 것

SQL 문장 쓰는 법

끝에 ;를 붙여야 실행돼요

SQL 낱말은 대문자로 쓰는 게 약속이에요.
소문자도 돼요

표와 데이터베이스 이름은 소문자로 써요

CREATE DATABASE로 만들고 USE로 들어가요

```
mysql> CREATE DATABASE shop;
Query OK, 1 row affected (0.00 sec)

mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| cafe               |
| information_schema |
| mysql              |
| performance_schema |
| shop               |
| sys                |
+--------------------+
6 rows in set (0.00 sec)

mysql> USE shop;
Database changed
mysql> SHOW TABLES;
Empty set (0.00 sec)
```

shop을 만들었어요

새로 생긴 shop

이제 shop 안에서 쳐요

아직 표가 없어요



mysql, sys처럼 MySQL이 스스로 쓰는 데이터베이스는 그림에서 뺐어요.

CREATE TABLE로 빈 표를 만들어요

```
~ $ docker exec -it db mysql -u root -p cafe
…
mysql> CREATE TABLE orders (id INT PRIMARY KEY, menu VARCHAR(20), qty INT);
Query OK, 0 rows affected (0.01 sec)
```

끝에 cafe를 적으면 cafe에 바로 들어가요

```
CREATE TABLE orders (
    id INT PRIMARY KEY,
    menu VARCHAR(20),
    qty INT
);
```

- `CREATE TABLE orders (` — orders라는 표를 만들어요
- `id INT PRIMARY KEY,` — 정수 · 기본 키
- `menu VARCHAR(20),` — 20자까지의 글자
- `qty INT` — 정수
- `);`

orders

id	menu	qty
INT	VARCHAR(20)	INT

열 이름과 타입만 있고 행은 아직 0개예요.

한 줄로 쳐도 되고, 여러 줄로 나눠 쳐도 돼요. 쉼표로 열을 나눠요.

SHOW TABLES로 표 목록을, DESCRIBE로 열을 봐요

```
mysql> SHOW TABLES;
Empty set (0.00 sec)        ← 아직 없어요

mysql> CREATE TABLE orders (…);
Query OK, 0 rows affected (0.01 sec)

mysql> SHOW TABLES;
+----------------+
| Tables_in_cafe |
+----------------+
| orders         |        ← 생겼어요
+----------------+
1 row in set (0.00 sec)
```

```
mysql> DESCRIBE orders;
+-------+-------------+------+-----+---------+-------+
| Field | Type        | Null | Key | Default | Extra |
+-------+-------------+------+-----+---------+-------+
| id    | int         | NO   | PRI | NULL    |       |
| menu  | varchar(20) | YES  |     | NULL    |       |
| qty   | int         | YES  |     | NULL    |       |
+-------+-------------+------+-----+---------+-------+
3 rows in set (0.01 sec)
```

Field 열 이름

Type 열의 타입 · 32에서 본 INT, VARCHAR

Null 비워 둬도 되는지 · 기본 키는 NO

Key PRI는 기본 키(PRIMARY KEY)

INSERT INTO로 행을 한 줄씩 넣어요

```
mysql> INSERT INTO orders VALUES (1, 'latte', 2);
Query OK, 1 row affected (0.01 sec)
mysql> INSERT INTO orders VALUES (2, 'americano', 1);
Query OK, 1 row affected (0.00 sec)
```

1행이 들어갔어요

orders

id	menu	qty
1	latte	2
2	americano	1

값은 표를 만들 때 적은 열 순서대로 넣어요. id, menu, qty예요.

글자는 `'latte'`처럼 작은따옴표로 감싸고, 숫자는 그대로 써요.

SELECT로 표를 꺼내 봐요

```
mysql> SELECT * FROM orders;
+----+-----------+------+
| id | menu      | qty  |
+----+-----------+------+
|  1 | latte     |    2 |
|  2 | americano |    1 |
+----+-----------+------+
2 rows in set (0.00 sec)
```

* = 모든 열

```
mysql> SELECT menu, qty FROM orders;
+-----------+------+
| menu      | qty  |
+-----------+------+
| latte     |    2 |
| americano |    1 |
+-----------+------+
2 rows in set (0.00 sec)
```

적은 열만 꺼내요

FROM 뒤에 어느 표에서 꺼낼지 적어요. 조건으로 고르는 WHERE, 고치는 UPDATE, 지우는 DELETE는 다음에 배워요.

DROP으로 표와 데이터베이스를 지워요

```
~ $ docker exec -it db mysql -u root -p shop
Enter password:
…
mysql> CREATE TABLE test (id INT);
Query OK, 0 rows affected (0.00 sec)

mysql> SHOW TABLES;
+----------------+
| Tables_in_shop |
+----------------+
| test           |
+----------------+
1 row in set (0.01 sec)

mysql> DROP TABLE test;
Query OK, 0 rows affected (0.01 sec)

mysql> SHOW TABLES;
Empty set (0.00 sec)
```

test 표를 지워요

표가 없어졌어요

```
mysql> DROP DATABASE shop;
Query OK, 0 rows affected (0.00 sec)

mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| cafe               |
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.00 sec)
```

shop을 통째로

shop이 없어요

DROP은 묻지 않고 바로 지우고, 되돌릴 수 없어요.

안에 든 행까지 다 사라져요. 연습할 때는 지워도 되는 것에만 쳐요.

3. 계정과 접속 방법

-e 두 개를 더하면 앱이 쓸 계정이 생겨요

```
~ $ docker exec -it db mysql -u cafe -p cafe
Enter password:
…
mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| cafe               |
| information_schema |
| performance_schema |
+--------------------+
3 rows in set (0.00 sec)

mysql> CREATE TABLE orders (id INT PRIMARY KEY, menu VARCHAR(20), qty INT);
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE DATABASE shop;
ERROR 1044 (42000): Access denied for user 'cafe'@'%' to database 'shop'
```

cafe5678을 쳐요

cafe 하나만 보여요

docker run에 더한 -e 두 개

`-e MYSQL_USER=cafe`

`-e MYSQL_PASSWORD=cafe5678`

cafe 사용자를 만들고, MYSQL_DATABASE의 cafe만 쓰게 해 줘요.

	root	cafe
비밀번호	cafe1234	cafe5678
보이는 DB	전부	cafe만
새 DB 만들기	돼요	막혀요

앱에는 root 대신 cafe 계정을 줘요. 실수해도 cafe 밖은 건드리지 못해요.

3. 계정과 접속 방법

bash로 먼저 들어가서 mysql을 열어도 돼요

```
~ $ docker exec -it db bash
bash-5.1# mysql -u root -p cafe
Enter password:
…

mysql> SHOW TABLES;
+----------------+
| Tables_in_cafe |
+----------------+
| orders         |
+----------------+
1 row in set (0.01 sec)

mysql> exit
Bye
bash-5.1# exit
exit
```

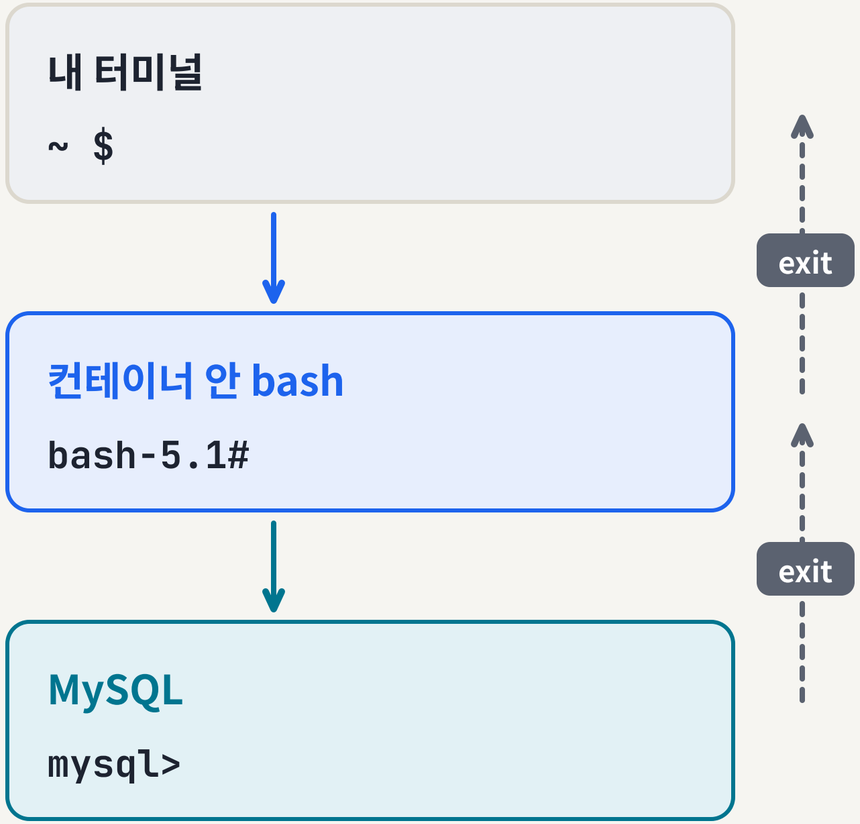
컨테이너 안 bash로

그 안에서 mysql

SQL을 쳐요

mysql에서 나와요

bash에서도 나와요



컨테이너 안의 파일도 같이 볼 때 써요. 나올 때는 exit를 두 번 쳐요.

3. 계정과 접속 방법

-e로 SQL 한 문장만 실행하고 바로 나와요

```
~ $ docker exec db mysql -u root -pcafe1234 cafe -e "SHOW TABLES;"
Tables_in_cafe
orders
mysql: [Warning] Using a password on the command line interface can be insecure.
```

결과만 받고 끝나요

결과를 받기만 하니 키보드로 칠 일이 없어서 -it를 빼요. 31의 docker exec 이름 명령과 같아요.

```
~ $ docker exec -it db mysql -u root -pcafe1234 cafe
mysql: [Warning] Using a password on the command line interface can be insecure.
…
mysql> exit
Bye
```

비밀번호를 묻지 않아요

-p 바로 뒤에 띄우지 않고 붙이면 비밀번호를 묻지 않아요

비밀번호가 명령 기록에 남아서 Warning이 나와요. 연습할 때만 써요.

언제 어떤 방법?

`docker exec -it db mysql -u root -p`

SQL을 여러 개 칠 때 · 기본

`docker exec -it db bash`

컨테이너 안도 같이 볼 때

`docker exec db mysql … -e "SQL"`

한 문장만 확인할 때

4. 볼륨과 compose

볼륨 없이 컨 컨테이너를 지우면 표도 같이 사라져요

```
~ $ docker stop db
db
~ $ docker rm db                                  앞 장까지 쓰던 db를 지우고
db
~ $ docker run -d --name db -e MYSQL_ROOT_PASSWORD=cafe1234 -e MYSQL_DATABASE=cafe -p 3306:3306 mysql:8.4
c7cd14188b94…
~ $ docker exec -it db mysql -u root -p cafe
Enter password:
…
mysql> SELECT * FROM orders;
ERROR 1146 (42S02): Table 'cafe.orders' doesn't exist
```

orders 표가 없대요. 표 안의 주문도 같이 사라졌어요.

cafe 데이터베이스는 -e로 다시 만들어졌지만, 그 안은 텅 비어 있어요.

4. 볼륨과 compose

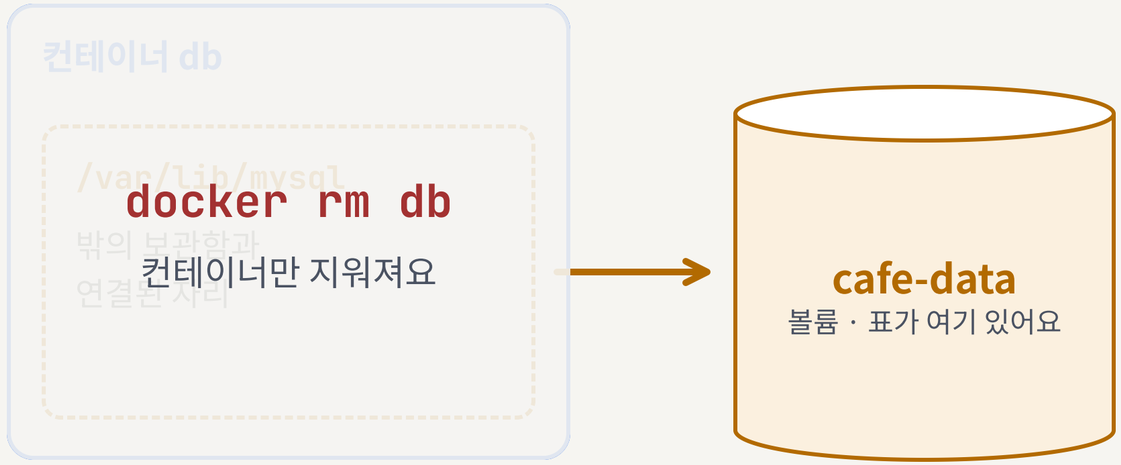
표는 컨테이너 안 /var/lib/mysql 폴더에 저장돼요

볼륨 없이



30에서 본 것처럼, 컨테이너 안에서 만든 파일은 컨테이너와 운명을 같이해요.

볼륨을 붙이면



새 컨테이너에 같은 볼륨을 붙이면 예전 표를 그대로 읽어요.

---

mysql 이미지는 -v를 안 붙여도 이름 없는 볼륨을 스스로 만들어요. 다만 새 컨테이너는 새 이름 없는 볼륨을 받아서 예전 표를 못 찾아요. docker volume ls에 긴 이름이 쌓이는 게 이 볼륨이에요.

4. 볼륨과 compose

-v cafe-data:/var/lib/mysql을 붙이면 지웠다 켜도 남아요

```
docker run … -p 3306:3306 -v cafe-data:/var/lib/mysql mysql:8.4
```

이름 있는 볼륨을 MySQL 데이터 폴더에

```
~ $ docker run … -v cafe-data:/var/lib/mysql mysql:8.4
13cac4776b85…
~ $ docker exec -it db mysql -u root -p cafe
…
mysql> CREATE TABLE orders (…);
Query OK, 0 rows affected
mysql> INSERT INTO orders VALUES (1, 'latte', 2);
Query OK, 1 row affected (0.01 sec)
~ $ docker stop db
db
~ $ docker rm db
db
```

주문을 넣고 컨테이너를 지워요

```
~ $ docker run … -v cafe-data:/var/lib/mysql mysql:8.4
501e0baabb23…
~ $ docker exec -it db mysql -u root -p cafe
…

mysql> SELECT * FROM orders;
+----+-------+------+
| id | menu  | qty  |
+----+-------+------+
|  1 | latte |    2 |
+----+-------+------+
1 row in set (0.00 sec)
```

같은 볼륨으로 다시 켜면 주문이 그대로예요

4. 볼륨과 compose

길어진 docker run을 compose.yaml에 옮겨 적어요

30에서 쓴 ports, volumes에 **environment**가 하나 더 생겼어요.
-e 자리예요.

```
-e MYSQL_ROOT_PASSWORD=cafe1234
-e MYSQL_DATABASE=cafe

-p 3306:3306

-v cafe-data:/var/lib/mysql
```

맨 아래 volumes:는 이 파일에서 쓸
이름 있는 볼륨을 적는 칸이에요.

~/cafe-db/compose.yaml

```yaml
services:
  db:
    image: mysql:8.4
    environment:
      MYSQL_ROOT_PASSWORD: cafe1234
      MYSQL_DATABASE: cafe
    ports:
      - "3306:3306"
    volumes:
      - cafe-data:/var/lib/mysql

volumes:
  cafe-data:
```

4. 볼륨과 compose

docker compose up -d로 켜고, 서비스 이름 db로 들어가요

```
~/cafe-db $ docker compose up -d
[+] up 3/3
 ✔ Network cafe-db_default  Created                                0.0s
 ✔ Volume cafe-db_cafe-data Created                                0.0s
 ✔ Container cafe-db-db-1   Started                                0.1s

~/cafe-db $ docker compose ps
NAME           IMAGE       COMMAND                  SERVICE   CREATED          STATUS         PORTS
cafe-db-db-1   mysql:8.4   "docker-entrypoint.s…"   db        10 seconds ago   Up 9 seconds   0.0.0.0:3306->3306/tcp, 33060/tcp

~/cafe-db $ docker compose exec db mysql -u root -p cafe
Enter password:
…
mysql>
```

폴더 이름이 앞에 붙은 볼륨

컨테이너 이름 대신 서비스 이름

`docker compose exec 서비스` 명령은 31의 docker exec와 같은 일을 해요.

4. 볼륨과 compose

30, 31에서 쓴 컨테이너와 무엇이 같고 다를까요

	30, 31 · nginx	33 · MySQL
설정 건네기	설정 파일을 -v ./conf로	**-e 환경 변수로**
볼륨에 담는 것	./site의 html 파일 (내가 고쳐요)	**/var/lib/mysql 표 파일 (MySQL만 고쳐요)**
들어가기	`docker exec -it web bash`	**`docker exec -it db mysql -u root -p`**
켜졌는지 보기	`curl localhost`	**logs의 port: 3306, SHOW DATABASES;**
기다리는 포트	`80`	**`3306`**

run, ps, logs, stop, rm, exec, compose는 그대로예요. 이미지에 따라 옵션만 달라져요.

5. 자주 나는 오류

비밀번호 -e를 빠뜨리면 켜지자마자 꺼져요

```
~ $ docker run -d --name db mysql:8.4
f0cee30f63a8…
~ $ docker ps -a
CONTAINER ID   IMAGE       COMMAND                  CREATED         STATUS                     PORTS     NAMES
f0cee30f63a8   mysql:8.4   "docker-entrypoint.s…"   4 seconds ago   Exited (1) 3 seconds ago             db
~ $ docker logs db
…
[ERROR] [Entrypoint]: Database is uninitialized and password option is not specified
    You need to specify one of the following as an environment variable:
    - MYSQL_ROOT_PASSWORD
    - MYSQL_ALLOW_EMPTY_PASSWORD
    - MYSQL_RANDOM_ROOT_PASSWORD
```

`docker rm db`로 꺼진 컨테이너를 지우고, -e MYSQL_ROOT_PASSWORD를 붙여 다시 켜요.

5. 자주 나는 오류

켜지자마자 들어가면 아직 준비 중이라 접속이 안 돼요

```
~ $ docker run -d --name db -e MYSQL_ROOT_PASSWORD=cafe1234 -e MYSQL_DATABASE=cafe -p 3306:3306 mysql:8.4
f761cbba159d…
~ $ docker exec -it db mysql -u root -p
Enter password:
ERROR 2002 (HY000): Can't connect to local MySQL server through socket '/var/run/mysqld/mysqld.sock' (2)
```

ERROR 2002는 MySQL 서버를 못 찾았다는 뜻이에요. 처음 켤 때 10초쯤 준비하는 동안 나와요.

docker logs db에서 마지막 ready for connections 줄에 port: 3306이 보일 때 다시 들어가요.

port: 3306

port: 0이 나온 줄은 임시 서버라서 아직이에요.

5. 자주 나는 오류

비밀번호를 잘못 치면 Access denied가 나와요

```
~ $ docker exec -it db mysql -u root -p
Enter password:
ERROR 1045 (28000): Access denied for user 'root'@'localhost' (using password: YES)
```

cafe123을 쳤어요. 4가 빠졌어요

화면에 안 보여서 잘못 치기 쉬워요	천천히 다시 쳐요
`using password: YES`	비밀번호를 받긴 했는데 틀렸다는 뜻이에요
기억이 안 나면	**`docker exec db env`**로 MYSQL_ROOT_PASSWORD를 봐요

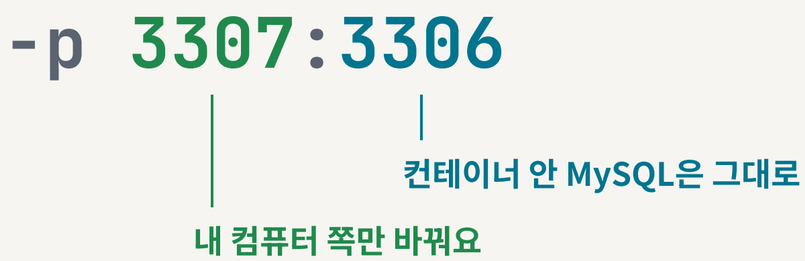
5. 자주 나는 오류

3306번을 이미 쓰고 있으면 3307:3306으로 바꿔요

```
~ $ docker run -d --name db2 -e MYSQL_ROOT_PASSWORD=x -p 3306:3306 mysql:8.4
dd81d7a75daafa42dd5a3510f000873f04ce51e574ba75bb7220f54edaf0f3f0
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on
endpoint db2 (f15e51cca32ad170b0b5c5b5d23b9d21891b4817dc2933f0308ddc8af4289260): Bind for 0.0.0.0:3306 failed: port is already
allocated

Run 'docker run --help' for more information
```

앞의 db가 3306번을 쓰는 중이라 db2가 못 켜졌어요. 내 컴퓨터에 MySQL을 직접 설치해 둬도 같은 오류가 나요.



먼저 docker ps로 3306번을 쓰는 컨테이너가 있는지 보고, 안 쓰는 것이면 꺼요.

5. 자주 나는 오류

SQL을 칠 때 자주 하는 실수 두 가지예요

1 끝에 ;를 빠뜨렸어요

```
mysql> SELECT * FROM orders
    -> ;    ← 아직 안 끝났대요

+----+-----------+------+
| id | menu      | qty  |
…
```

->가 나오면 ;만 치고 Enter를 눌러요.

2 표 이름을 잘못 썼어요

```
Enter password:
ERROR 1064 (42000): You have an error in your SQL
syntax; check the manual that corresponds to your
MySQL server version for the right syntax to use
near 'order' at line 1
```

5. 자주 나는 오류

볼륨이 남아 있으면 바꾼 비밀번호가 적용되지 않아요

```
~ $ docker run -d --name db -e MYSQL_ROOT_PASSWORD=new5678 -e MYSQL_DATABASE=cafe -p 3306:3306 -v cafe-data:/var/lib/mysql mysql:8.4
c28ab4e5f1d5…
~ $ docker exec -it db mysql -u root -p
Enter password:
ERROR 1045 (28000): Access denied for user 'root'@'localhost' (using password: YES)
```

new5678을 쳤어요

```
~ $ docker logs db
[Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.13-1.el9 started.
[Note] [Entrypoint]: Switching to dedicated user 'mysql'
[Note] [Entrypoint]: Entrypoint script for MySQL Server 8.4.13-1.el9 started.
…
```

Initializing, Creating database 줄이 없어요. 볼륨에 이미 데이터가 있어서 처음 준비를 건너뛰었어요.

예전 비밀번호 cafe1234로 들어가요

표가 그대로 있어요

처음부터 다시: `docker volume rm cafe-data`

표도 같이 지워져요

6. 실제 상황 예제

예제 1 · 오늘 문을 닫고 내일 열어도 어제 주문이 남아요

월요일

```
~/cafe-db $ docker compose up -d
⋯
~/cafe-db $ docker compose exec db mysql -u root -p cafe
⋯
mysql> CREATE TABLE orders (…);
mysql> INSERT INTO orders VALUES (1, 'latte', 2);
mysql> INSERT INTO orders VALUES (2, 'cake', 1);
mysql> exit
~/cafe-db $ docker compose down
⋯
```

화요일

```
~/cafe-db $ docker compose up -d
⋯
~/cafe-db $ docker compose exec db mysql -u root -p cafe
⋯

mysql> INSERT INTO orders VALUES (3, 'americano', 3);
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM orders;
+----+-----------+------+
| id | menu      | qty  |
+----+-----------+------+
|  1 | latte     |    2 |
|  2 | cake      |    1 |
|  3 | americano |    3 |
+----+-----------+------+
3 rows in set (0.00 sec)
```

월요일 주문

오늘 주문

down은 컨테이너만 지우고 볼륨 cafe-db_cafe-data는 남겨 둬요.

6. 실제 상황 예제

예제 2 · 처음 켤 때 메뉴 표를 SQL 파일로 채워요

~/cafe-db/init/menu.sql

```
CREATE TABLE menu (id INT PRIMARY KEY, name VARCHAR(20), price INT);
INSERT INTO menu VALUES (1, 'americano', 4000);
INSERT INTO menu VALUES (2, 'latte', 4500);
INSERT INTO menu VALUES (3, 'cake', 5000);
```

compose.yaml

```
    volumes:
      - cafe-data:/var/lib/mysql
      - ./init:/docker-entrypoint-initdb.d
```

```
~/cafe-db $ docker compose down -v          볼륨까지 지워요
…
~/cafe-db $ docker compose up -d
…
~/cafe-db $ docker compose logs db
…
… /usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/menu.sql
…
~/cafe-db $ docker compose exec db mysql -u root -p cafe
…

mysql> SELECT * FROM menu;
+----+-----------+-------+
| id | name      | price |
+----+-----------+-------+
|  1 | americano |  4000 |
|  2 | latte     |  4500 |
|  3 | cake      |  5000 |
+----+-----------+-------+
3 rows in set (0.00 sec)
```

볼륨이 남아 있으면 안 돌아요

예제 1의 볼륨에서 up만 했더니 SHOW TABLES;에 orders만 있었어요. 그래서 down -v로 볼륨을 지우고 켰어요. 쌓인 주문도 같이 지워져요.

./init 폴더의 .sql 파일은 볼륨이 빈 채로 처음 켤 때 한 번만 실행돼요.

새 팀원도 git clone 뒤 up -d 한 번이면 같은 메뉴 표를 받아요.

정리

막히면 이렇게 풀어요

이런 글이 보이면	이렇게 해요
`Exited (1) · You need to specify …` 비밀번호 -e를 빠뜨렸어요	**docker rm 뒤 -e를 붙여 다시**
`ERROR 2002 · Can't connect` 서버가 아직 준비 중이에요	**logs에서 port: 3306 보기**
`ERROR 1045 · Access denied` 비밀번호가 틀렸어요	**천천히 다시, 볼륨 함정 확인**
`port is already allocated` 3306번을 다른 것이 써요	**"3307:3306"으로 바꾸기**
`-> · ERROR 1064` ; 빠짐, SQL 문법 오류	**; 치기, near 뒤를 보기**
`ERROR 1146 · doesn't exist` 표가 없어요. 볼륨 없이 지웠나요	**-v 이름:/var/lib/mysql**

정리

한 장 요약

켜고 들어가기

`-e MYSQL_ROOT_PASSWORD=…`

비밀번호 · 꼭

`-e MYSQL_DATABASE=cafe`

처음 만들 데이터베이스

`docker logs db`

port: 3306이면 준비 끝

`docker exec -it db mysql -u root -p cafe`

접속, 나올 땐 exit

SQL

`CREATE DATABASE shop; · USE shop;`

데이터베이스 만들고 들어가기

`CREATE TABLE 표 (열 타입, …);`

빈 표 · SHOW TABLES, DESCRIBE로 확인

`INSERT INTO 표 VALUES (…);`

행 넣기 · SELECT로 꺼내기

`DROP TABLE 표; · DROP DATABASE 이름;`

지우기 · 되돌릴 수 없어요

남기기

`-v cafe-data:/var/lib/mysql`

지워도 남는 표

`compose.yaml environment`

-e 자리

`docker compose down`

볼륨은 남겨요

`docker compose down -v`

볼륨까지 지워요 · 연습할 때만

데이터베이스는 이미지 하나, 환경 변수 몇 개, 볼륨 하나로 켜고 지켜요.

34에서는 같은 순서로 PostgreSQL을 켜고 둘을 비교해요.